

AI Usage DISCLOSURE FORM

1. Team Details

Team Name: Cheesecake

Project / Product Name: FlowPilot — Learn-Once, Replay-Anywhere Teachable Voice Automation for Android

Organization / Institution (if any): SRM Institute of Science and Technology (SRMIST), Kattankulathur

Submission Date: 30 September 2026

2. AI Usage Declaration

Did your team use any Artificial Intelligence (AI) in developing this project? Yes / No

Yes — AI was used strictly and only where required for the core product functionality (voice-command semantic matching, flow generalisation, and speech-to-text). All other components — including the entire Android Accessibility Execution Engine, UI, backend architecture, and security modules — were designed and coded entirely by the team without AI code generation.

3. Purpose of AI Usage (Brief Details)

Idea generation / brainstorming: Evaluated Accessibility Tree abstractions and parameter-slot generalisation strategies for Android voice assistants. AI was used only to explore high-level architecture ideas; all final design decisions were made by the team.

Code generation or assistance: Limited AI assistance for initial boilerplate only: Retrofit HTTP client stubs, FastAPI route scaffolds, and benchmark test harness structure. All generated code was thoroughly reviewed, modified, tested, and integrated by the team. The core Accessibility Engine (Kotlin) was written entirely by the team without AI assistance.

UI / UX design: Minor assistance in selecting Material 3 colour tokens and Jetpack Compose component styling. All final UI decisions and implementation done by the team.

Content creation: Architecture diagrams, sequence flows, and API documentation formatting. All content was verified for accuracy against the actual codebase.

Data analysis: Assistance in structuring the 25-query, 5-app evaluation dataset and latency-distribution calculations for the benchmark suite.

Testing / debugging: Assistance in drafting unit test assertions for vector matching and slot compilation (test_flowpilot.py). All tests were reviewed and verified by the team.

Other (AI in the product itself): AI models are embedded strictly where required for product functionality: Gemini 2.0 Flash for flow compilation, Sentence-BERT + ChromaDB for semantic intent

matching, and Whisper / Android SpeechRecognizer for speech-to-text. No AI model receives passwords, PINs, OTPs, or card details at any point.

4. Feature Origin Classification (Please add/delete based on the number of the feature)

1. Feature Name: Generalised Flow Compilation (LLM Flow Synthesis)

2. Self-Generated/AI-Generated/Both: Both

3. Description (include AI tools/platform Used, prompt used, Output Summary, Modification): AI tool: Google Gemini (gemini-2.0-flash; fallbacks gemini-1.5-flash, gemini-2.0-flash-lite) via the google-genai Python SDK. Prompt: A structured “FlowCompiler” system prompt that receives the raw UI action trace (JSON array from the AccessibilityService) and instructs the model to: (a) clean noise and duplicate events, (b) abstract UI elements into semantic selectors, (c) convert user-typed values into named parameter slots with regex extractors, (d) generate 3–5 trigger phrases for voice matching, (e) flag payment/OTP/password steps as auth pauses, (f) estimate per-step wait times, and return strict JSON. Output: A generalised, parameterised FlowGraph with steps, slots, trigger phrases, and auth flags. Modification (student work): Team designed the system prompt, added Pydantic schema validation with strict type checking, implemented retry logic with model fallback chain, and built a deterministic heuristic fallback compiler for offline use when Gemini is unavailable.

1. Feature Name: Voice Command Semantic Matching

2. Self-Generated/AI-Generated/Both: Both

3. Description (include AI tools/platform Used, prompt used, Output Summary, Modification): AI tool: Pre-trained Sentence-BERT model all-MiniLM-L6-v2 (384-dimensional embeddings, no fine-tuning) with a ChromaDB cosine-similarity vector index. Prompt: Not applicable — the transcribed voice command text is the direct input to the embedding model. Output: Ranked list of matching flows with confidence scores, enabling paraphrase-robust matching (e.g., “Order a Margherita” ↔ “Get me pizza from Dominos”). Modification (student work): Team wrote the entire matching service architecture, de-duplication logic across trigger phrases, confidence cutoff threshold (cosine distance ≤ 0.45) for rejecting unlearned intents, ambiguity resolution routing, and the “Would you like to teach me?” fallback dialogue.

1. Feature Name: Voice Speech-to-Text (STT)

2. Self-Generated/AI-Generated/Both: AI-Generated (pre-trained models, integrated by the team)

3. Description (include AI tools/platform Used, prompt used, Output Summary, Modification): AI tool: OpenAI Whisper (“base” model, run locally on the backend server) and Android native SpeechRecognizer API. Prompt: Not applicable — audio input is processed directly. Output: Text transcript of the spoken voice command. Modification (student work): Team wrote all integration code including the audio capture endpoint, backend Whisper service wrapper, on-device Android SpeechRecognizer fallback path, and the routing logic to select the appropriate STT engine.

1. Feature Name: Android Accessibility Execution Engine (Record & Replay)

2. Self-Generated/AI-Generated/Both: Self-Generated (100% student-engineered — no AI used)

3. Description (include AI tools/platform Used, prompt used, Output Summary, Modification): AI tool: None. No AI tool or prompt was used for this component. This is entirely custom Kotlin code built on Android's AccessibilityService API. It includes: (1) FlowRecorderService — captures UI events and accessibility node metadata during user demonstration; (2) FlowReplayService — replays steps by locating elements via a 4-tier cascading finder (resource-id → content-description → text-match → bounding-box gesture fallback); (3) Dynamic parameter injection — substitutes slot values at runtime; (4) Overlay dismissal heuristic — auto-dismisses promo pop-ups and unexpected dialogs; (5) Stuck detection — halts safely within 30 seconds if a step cannot be completed; (6) Security Auth Pause Gate — freezes execution on payment/OTP/PIN/password screens and hands control to the user. This component represents the core technical innovation of FlowPilot and was designed and implemented entirely by the team.

5. Ethical & Compliance Confirmation

AI usage complies with guidelines and policies. Yes

No proprietary or copyrighted data misused. I Agree

Zero Credential Capture Guarantee: No passwords, PINs, OTPs, or payment card details are captured, processed, or logged by any AI model or server component at any point.

6. Declaration & Sign-Off

Name of Team Representative: Sanjeev Aryan

Role: Team Lead, Team Cheesecake

Email: aryansanjeev0651@gmail.com

Signature: *Sanjeev Aryan* (electronically signed)

Date: 30 September 2026

This form is submitted as part of the official FlowPilot hackathon submission package for Samsung PRISM GenAI Hackathon 3rd Edition (2026-27), Theme #03: Teachable Voice Automation.